NAME :TARANG KATHIRIYA
ADMISSSION NO:U23CS148
ASSIGNMENT 2 SYSTEM SOFTWARE

Lab Assignment – 2

1. Write a C Program to recognize a valid arithmetic expression and identify the identifiers and operators present. Print them separately.
2. Write a C program to identify whether a given line is a comment or not.
3. Create Assembler as discussed in Class.

1.

```
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
   string expr;
   string identifiers = "";
   string operators = "";

   bool expectIdentifier = true;
   bool isValid = true;

   cout << "Enter simple arithmetic expression (e.g., a + b * c): ";
   getline(cin, expr);

   for (int i = 0; i < expr.length(); i++) {
       char c = expr[i];


       if (c == ' ') continue;


       if (isalpha(c)) {
           if (!expectIdentifier) {
```

```
                isValid = false;
                break;
            }
            identifiers += c;
            identifiers += " ";
            expectIdentifier = false;
        }


        else if (c == '+' || c == '-' || c == '*' || c == '/' || c == '%')
{
            if (expectIdentifier) {

                isValid = false;
                break;
            }
            operators += c;
            operators += " ";
            expectIdentifier = true;
        }


        else {
            isValid = false;
            break;
        }
    }


    if (expectIdentifier) {
        isValid = false;
    }

    cout << "\n----------------------------\n";
    if (isValid) {
        cout << "Status: Valid Expression\n";
        cout << "Identifiers: " << identifiers << endl;
        cout << "Operators:   " << operators << endl;
    } else {
```

```
        cout << "Status: Invalid Expression" << endl;
    }
    cout << "----------------------------\n";

    return 0;
}
```

OUTPUT:

```
(base) administrator@administrator-HP-ProDesk-400-G7-Small-Form-Factor-PC:~/u23cs148 sytem software/lab 2$ cd "/home/adminis
trator/u23cs148 sytem software/lab 2/" && g++ 1.cpp -o 1 && "/home/administrator/u23cs148 sytem software/lab 2/"1
Enter simple arithmetic expression (e.g., a + b * c): a  b

---------------------------
Status: Invalid Expression
---------------------------
(base) administrator@administrator-HP-ProDesk-400-G7-Small-Form-Factor-PC:~/u23cs148 sytem software/lab 2$ cd "/home/adminis
trator/u23cs148 sytem software/lab 2/" && g++ 1.cpp -o 1 && "/home/administrator/u23cs148 sytem software/lab 2/"1
Enter simple arithmetic expression (e.g., a + b * c): a   + b

---------------------------
Status: Valid Expression
Identifiers: a b
Operators:   +
---------------------------
```

2.

```
#include <bits/stdc++.h>
#define MAX 166
using namespace std;
int main() {
    string str;
    cout<<"enter the line"<<endl;
    cin>>str;

    int i = 0;
    for (i = 0; i < str.size(); i++) {
        if (str[i] == ' ')
            continue;
        else
            break;
    }
    if (str[i] == '/' && str[i + 1] == '/') {
```

```
        printf("\nThis line is the comment\n");
    } else {
        printf("\nThis line is not the comment\n");
    }
    return 0;
}
```

OUTPUT:

```
(base) administrator@administrator-HP-ProDesk-400-G7-Small-Form-Factor-PC:~/u23cs148 sytem software/lab 2$ cd "/home/adminis
trator/u23cs148 sytem software/lab 2/" && g++ 2.cpp -o 2  && "/home/administrator/u23cs148 sytem software/lab 2/"2
enter the line
TARANG KATHIRIYA

This line is not the comment
(base) administrator@administrator-HP-ProDesk-400-G7-Small-Form-Factor-PC:~/u23cs148 sytem software/lab 2$ cd "/home/adminis
trator/u23cs148 sytem software/lab 2/" && g++ 2.cpp -o 2  && "/home/administrator/u23cs148 sytem software/lab 2/"2
enter the line
// TARANG KAHTIRIYA

This line is the comment
(base) administrator@administrator-HP-ProDesk-400-G7-Small-Form-Factor-PC:~/u23cs148 sytem software/lab 2$
```

3.

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <map>

using namespace std;
struct Line {
    string label;
    string opcode;
    string operand;
};

map<string, int> opcodeTable = {
    {"LOAD", 1},
    {"ADD", 2},
    {"STORE", 3},
    {"JMP", 4},
    {"DATA", 0}
};
```

```
map<string, int> symbolTable;

Line parseLine(string rawLine) {
   Line l;
   stringstream ss(rawLine);
   string token;

   if (ss >> token) {

       if (token.back() == ':') {
           token.pop_back();
           l.label = token;
           ss >> l.opcode;
       } else {
           l.opcode = token;
       }
       ss >> l.operand;
   }
   return l;
}

int main() {

   vector<string> sourceCode = {
       "START: LOAD A",
       "       ADD B",
       "       STORE C",
       "       JMP START",
       "A:     DATA 10",
       "B:     DATA 20",
       "C:     DATA 0"
   };

   vector<Line> parsedProgram;
   int LC = 0;

   cout << "--- PASS 1: SYMBOL TABLE GENERATION ---\n";

   for (const string& rawLine : sourceCode) {
```

```
        Line line = parseLine(rawLine);
        parsedProgram.push_back(line);

        if (!line.label.empty()) {

            symbolTable[line.label] = LC;
            cout << "Found Label: " << line.label << " at Address " << LC
<< endl;
        }

        if (!line.opcode.empty()) {
            LC++;
        }
    }

    cout << "\n--- PASS 2: CODE GENERATION ---\n";
    cout << "Addr\tOpcode\tOperand\n";
    cout << "----\t------\t-------\n";

    LC = 0;

    for (const Line& line : parsedProgram) {
        if (line.opcode.empty()) continue;

        int opValue = 0;
        int operandValue = 0;


        if (opcodeTable.find(line.opcode) != opcodeTable.end()) {
            opValue = opcodeTable[line.opcode];
        } else {
            cout << "Error: Unknown Opcode " << line.opcode << endl;
            continue;
        }


        if (line.opcode == "DATA") {

            operandValue = stoi(line.operand);
        }
```

```
        else if (!line.operand.empty()) {

            if (symbolTable.find(line.operand) != symbolTable.end()) {
                operandValue = symbolTable[line.operand];
            } else {
                cout << "Error: Undefined Label " << line.operand << endl;
            }
        }

        cout << LC << "\t" << opValue << "\t" << operandValue << endl;

        LC++;
    }

    return 0;
}
```

OUTPUT:

```
(base) administrator@administrator-HP-ProDesk
trator/u23cs148 sytem software/lab 2/" && g++
--- PASS 1: SYMBOL TABLE GENERATION ---
Found Label: START at Address 0
Found Label: A at Address 4
Found Label: B at Address 5
Found Label: C at Address 6

--- PASS 2: CODE GENERATION ---
Addr    Opcode  Operand
----    ------  -------
0       1       4
1       2       5
2       3       6
3       4       0
4       0       10
5       0       20
6       0       0
(base) administrator@administrator-HP-ProDesk
```